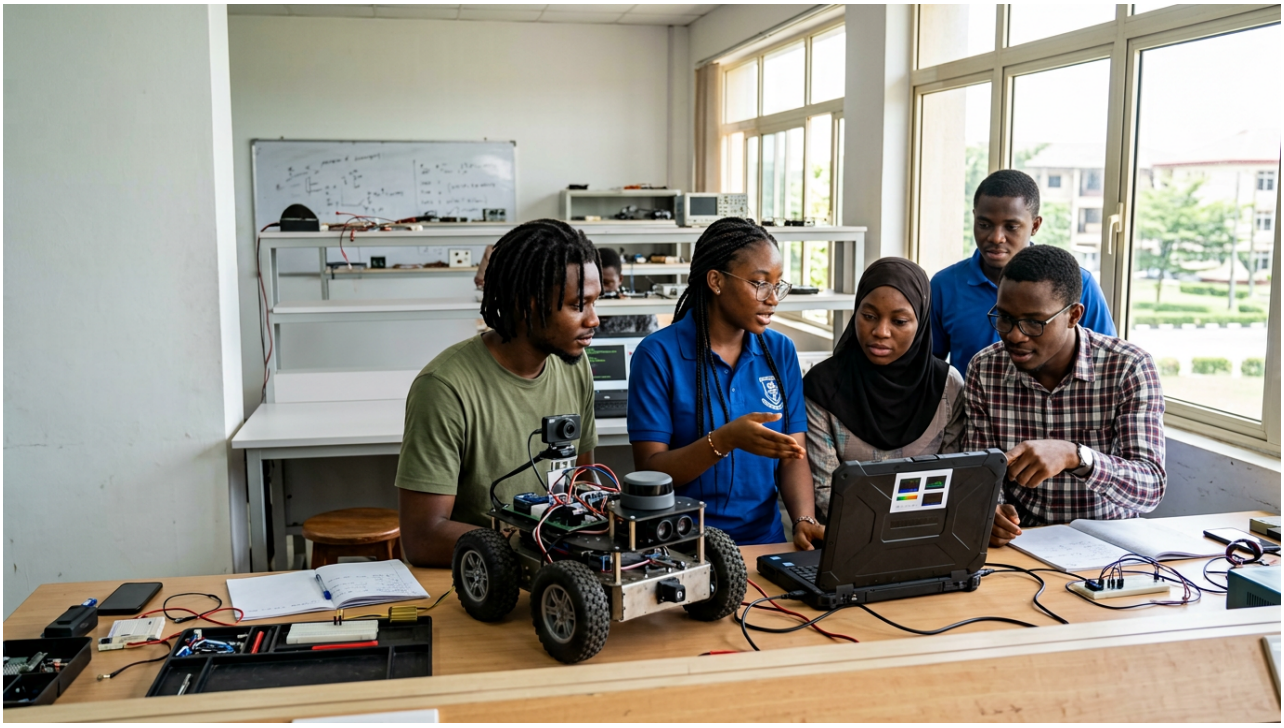
AI.101 — Introduction to Artificial Intelligence



Illustrative engineering-laboratory image; not documentary evidence of a specific university or event.

Narration manuscript · Module 3: Problem Formulation and Search

Academic level: Bachelor of Artificial Intelligence and Robotics Engineering
Content policy: No faith-related content.

1. Why problem formulation comes before search

Welcome back. In the previous module, we described an agent’s task environment using performance measures, environment, actuators and sensors. We now take one specific kind of task—finding a sequence of actions that reaches a goal—and express it as a search problem.

Search is often introduced as a collection of named algorithms: breadth-first search, depth-first search, uniform-cost search, A-star. But memorizing names is not the main goal. Search is a general strategy for exploring possible choices. The quality of a search result depends first on how we represent the world. If our states omit a fact that affects future actions, the algorithm may return a plan that cannot be executed. If we include irrelevant detail, the search may become unnecessarily large. Formulation is part of the engineering work.

Suppose a mobile robot needs to move from a laboratory to a reception area. A first model might represent each named corridor junction as a state and each traversable corridor as an action. That representation is useful if corridors are static and the robot can reliably traverse each one. It is inadequate if the robot must know whether a fire door is open, whether an elevator is available, or how much battery remains. Those facts may need to become part of the state.

A **state** is a representation of the information needed to make decisions about the future. It is not necessarily a complete copy of physical reality. A search algorithm can operate only on the state representation we provide. The designer must decide what information is necessary for predicting the consequences of actions and determining whether a goal has been reached.

2. The five components of a search problem

We will define a basic search problem using five elements.

The **initial state** describes where the process begins. For our robot, this might be “robot at laboratory junction, package secured, battery at 82 percent, route map version 4.” A useful initial state includes all details needed by the transition and goal models.

The **actions** describe choices available in a state. At a corridor junction, actions could be “take north corridor,” “take east corridor,” “wait,” or “request assistance.” We should not list actions that the robot cannot perform. If a doorway requires a credential the robot does not have, “open secure door” is not a valid action merely because it would be convenient.

The **transition model** describes what state results from taking an action. In a deterministic abstraction, taking the east corridor from junction A leads to junction B. In a stochastic model, the robot may arrive at B, fail because the corridor is blocked, or require a recovery action because wheel slip caused localization error. The transition model can be exact, estimated, learned or approximated, but its assumptions should be explicit.

The **goal test** determines whether the current state satisfies the task objective. “At reception” might not be enough. Perhaps the correct package must be delivered to an authorized staff member, the package must remain intact, and the handoff must be logged. A precise goal test avoids false success.

The **path cost** assigns a cost to a sequence of actions. It might represent distance, travel time, energy, risk exposure, or a combination. For a simple introductory problem, we may assign each corridor traversal a cost of one. For an engineering problem, different edges can have different costs. A ramp might take longer and consume more energy than a flat corridor. A crowded route might carry a higher risk cost. Costs need to be meaningful and consistent with the performance measure.

Show the five search problem components as an equation-like flow: initial state + actions + transition model → reachable state space; goal test selects acceptable states; path cost compares solution paths.

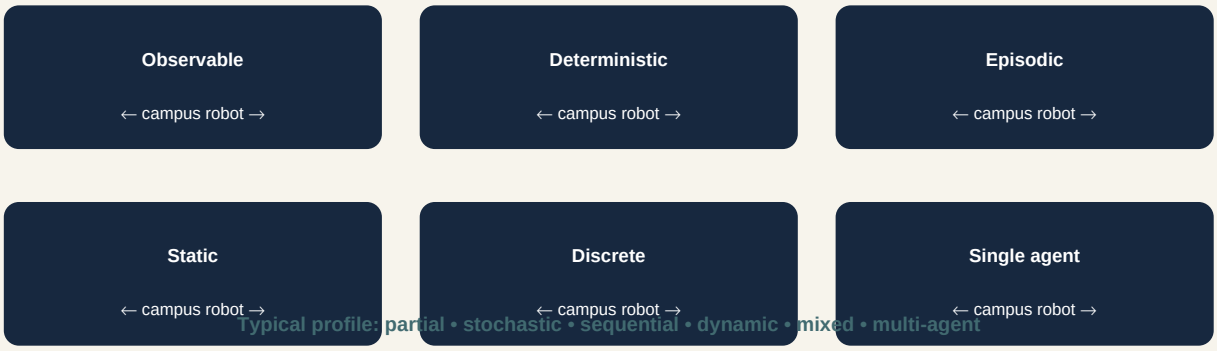


Diagram: Environment classification profile

The output of search is a **solution path**: a sequence of actions that reaches a goal. It is important to distinguish the path from the search process. Search may explore many states that are not on the final solution. A useful algorithm is judged not only by whether it can find a solution, but by how much time and memory it uses and whether the solution has desirable properties.

3. State spaces, graphs and search trees

A **state-space graph** represents states as nodes and actions as edges. If an action can move the robot from state A to state B, draw a directed edge from A to B. If the action works in both directions, the graph may contain edges in both directions. Edge labels can identify actions, and edge weights can represent costs.

A **search tree** is different. It records how the algorithm explores possibilities from a starting point. The same state can appear multiple times in a search tree if different action sequences reach it. In the graph of a building, a robot could arrive at the same junction by going north and then east or by going east and then north. These are two paths, but they end at one underlying location state—assuming no other relevant state variable differs.

This distinction matters because repeatedly exploring the same state can waste resources or create loops. Search algorithms often keep track of a frontier, which contains discovered but not yet expanded nodes, and an explored set, which contains nodes whose outgoing possibilities have already been considered. The exact bookkeeping depends on the algorithm.

Start with a graph containing a loop. Animate a naive search tree that revisits a location, then show graph search marking visited states to avoid redundant expansions.

The state definition affects whether two situations count as the same state. If the robot is at junction B with 80 percent battery and with 8 percent battery, are those states interchangeable? Not if the remaining route may require more energy than is available in the second case. If a door is closed in one situation and open in another, the future actions differ. A correct state representation must preserve distinctions that affect the available actions, transition outcomes or goal test.

At the same time, state detail has a cost. If we represent a robot’s exact position to the nearest micrometre, exact wheel temperature, every pixel of every camera image and every network packet, the state space becomes enormous. We need an abstraction suited to the task. For building-level route planning, named junctions may be enough. For collision avoidance near a person, finer spatial and temporal detail is needed. Different layers of a robot may use different state abstractions.

4. Breadth-first search

Breadth-first search expands the shallowest nodes first. It uses a queue: nodes discovered earliest are expanded first. If every action has equal cost, breadth-first search finds a path with the fewest actions, assuming a solution exists and the branching factor is finite.

Imagine a simple map with start S, two adjacent junctions A and B, and a goal G. Breadth-first search first explores states one action away from S, then states two actions away, then three. It does not get distracted by a long branch before checking all shorter routes. This is helpful when each step has equal cost and minimizing the number of steps is the actual objective.

But “fewest steps” is not always “cheapest” or “safest.” Suppose one path has two edges and requires crossing a crowded lobby, while another path has three edges through a quiet corridor. Breadth-first search treats the two edges as a better solution. If the task values safety, time or energy differently, we need a cost-sensitive method or a modified model.

Breadth-first search is complete under standard finite-branching assumptions: if a solution exists at some finite depth, it will eventually find one. It is optimal only when every action has the same cost. Its memory use can be large because the queue may contain many nodes at the boundary of the explored region. A search that is conceptually simple can become impractical when the number of alternatives grows rapidly.

5. Depth-first search

Depth-first search follows one branch as far as it can, then backtracks. It can be implemented with a stack or recursion. It may use much less memory than breadth-first search because it stores a narrower path and a smaller set of alternatives.

The trade-off is that depth-first search can spend a long time exploring an unhelpful branch. In a graph with cycles, it can loop forever if it does not track visited states or impose a depth limit. In an infinite-depth search space, it may never return to a shallow solution. It is not generally optimal: the first goal it reaches may be much more expensive than another route.

Consider a robot map where the north corridor leads through a long sequence of connected rooms, while the destination is only two junctions to the east. A depth-first search that chooses north first may explore many rooms before it backtracks and considers east. The result depends on action ordering. That is a reminder that algorithmic behavior can depend on seemingly minor implementation choices.

Depth-first methods are useful when memory is constrained, when solutions are expected deep in the search tree, or when the search can be safely bounded. They should not be selected merely because the code is short.

6. Uniform-cost search

Uniform-cost search expands the frontier node with the lowest accumulated path cost so far. Instead of treating every action as equal, it orders paths by their known cost from the start. A priority queue is commonly used.

Imagine three routes from a robot’s current location. Route A takes two short corridor segments, each with cost 5, for a total cost of 10. Route B takes three segments with costs 2, 2 and 2, for a total cost of 6. Route C takes a single segment through a restricted area with cost 20. Breadth-first search might prefer the one-edge route C because it has fewer steps. Uniform-cost search instead considers total cost and will prefer the route with cost 6, subject to finding a complete valid route.

If all edge costs are non-negative, uniform-cost search is complete under appropriate conditions and returns a least-cost solution. It may still expand many nodes, especially when many partial paths have similar low costs. Its correctness also depends on the cost model. If we assign an unrealistically low cost to a risky path, the search

can faithfully find the wrong route according to our model.

This is a central engineering lesson: an optimizer does not know whether its objective is sensible. It efficiently pursues the quantities we encode. Designers remain responsible for the meaning and consequences of those quantities.

7. A-star search and heuristic guidance

A-star search adds an estimate of the remaining cost to the goal. It ranks a path using the sum of the cost already paid and the estimated cost still to come. We write this as **f(n) = g(n) + h(n)**. Here, *g(n)* is the cost from the initial state to the current node, *h(n)* is the heuristic estimate from the current node to a goal, and *f(n)* is the estimated total cost of a solution path through that node.

A heuristic is useful information that guides search. For a robot moving on a simple grid, straight-line distance to the destination may be a lower-bound estimate of travel distance when obstacles can only make the route longer. A heuristic is **admissible** if it never overestimates the actual least remaining cost. Under standard conditions, A-star with an admissible heuristic and correct handling of repeated states can find an optimal path. In some graph-search formulations, consistency of the heuristic is also important for efficient and reliable behavior.

A heuristic that is too small may provide little guidance. If h(n) is zero for every state, A-star behaves like uniform-cost search. A more informative heuristic can reduce expansions. But a heuristic that overestimates may cause the algorithm to choose a suboptimal route. The designer must understand what guarantee is needed and whether the heuristic satisfies it.

For a real building, straight-line distance can be a weak guide if walls, locked doors and stairs constrain movement. A graph-distance estimate based on a simplified map may be more informative. If accessibility requirements prohibit stairs, the heuristic and transition model must respect that constraint. The mathematics is not separate from the requirements: a careless heuristic can encode an assumption about what routes are acceptable.

On a weighted map, show three competing routes. Display g(n) as known path cost, h(n) as remaining estimate and f(n) as their sum. Animate the frontier choice for uniform-cost search and A-star.]

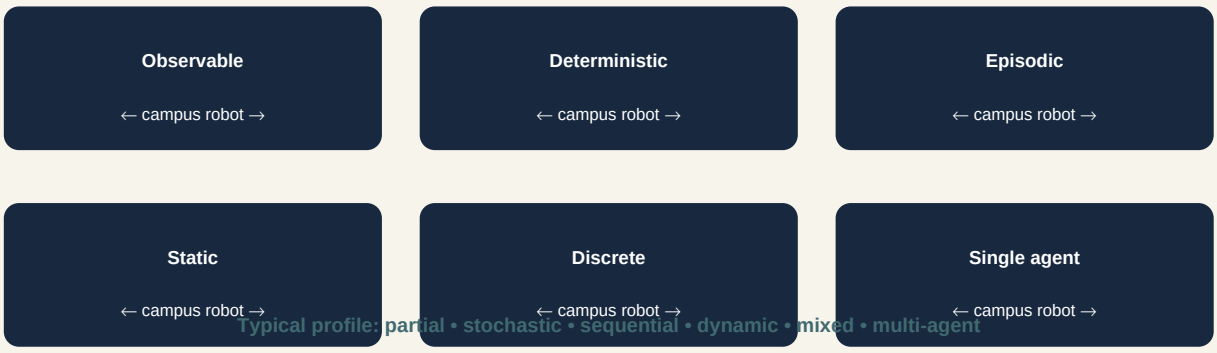


Diagram: Environment classification profile

8. Worked route example

Let us work a small example. The robot begins at S. It can reach A with cost 2 or B with cost 1. From A it can reach G with cost 5. From B it can reach C with cost 2, and C reaches G with cost 2. There is also a direct B-to-G edge costing 8.

Breadth-first search compares the number of edges. The path S-A-G has two edges, and S-B-G also has two edges. Depending on the order in which neighbors are considered, it may return either one. It does not distinguish their costs: the first route costs 7, the second costs 9.

Uniform-cost search considers cumulative cost. After expanding S, it places A at cost 2 and B at cost 1 in the priority queue. It expands B first, then discovers C at total cost 3 and G at total cost 9. It next expands A at cost 2 and discovers G at cost 7. It continues according to the frontier ordering and ultimately returns the least-cost path S-A-G, cost 7, unless it finds a still cheaper complete route. In this small graph, it does not stop merely because it has first generated a goal; it stops when the goal is selected for expansion from the lowest-cost frontier under the standard algorithm.

A-star can use a heuristic estimate. Suppose h(A)=4, h(B)=2 and h(C)=2, with h(G)=0. The estimated totals for A and B are both 6 and 3 respectively. A-star initially favors B. It may then discover C and G, and later compare their estimates with the alternative through A. The admissibility of these estimates is what preserves the optimality guarantee. If we invent estimates without checking them, a fast result may not be the best result.

This example demonstrates why we record both the search order and the final path cost. A route that looks shorter on a diagram can be more expensive according to the objective. A route that is found first is not necessarily optimal.

9. Scaling and practical engineering limits

Search can grow rapidly as the number of actions and the depth of possible solutions increase. If each state offers several choices, the number of paths may multiply at each step. This is called combinatorial growth. A method that is perfectly adequate on a classroom map may fail on a large building or a high-dimensional robot configuration space.

Engineers use several strategies to manage this. They choose a state representation that captures what matters without storing unnecessary detail. They use heuristics when valid estimates are available. They decompose the problem into levels, such as choosing a sequence of rooms before generating motor trajectories. They use constraints to eliminate impossible choices. They may also compute approximate solutions when exact optimality is too costly, provided the approximation is acceptable and its limitations are disclosed.

Search is also sensitive to changing environments. A route computed from a static map may become invalid when a corridor is blocked. The agent may need to monitor observations, update its internal model and replan. In a robot, planning and control operate on different time scales: a global planner may choose a corridor sequence, while a local controller continuously adjusts velocity and avoids nearby obstacles. The boundaries between these layers must be designed and tested.

10. Laboratory: implement and compare search

In the laboratory, represent a small building map as a weighted graph. Use at least eight named junctions, one start and one goal. Mark one corridor unavailable and assign different travel costs to the remaining edges. Implement breadth-first search, depth-first search, uniform-cost search, and A-star.

For each run, record the returned path, its total cost, number of expanded states, maximum frontier size, and whether the result is guaranteed to be optimal under the assumptions. Create a map visualization that highlights the expanded nodes and the final path. Use a consistent expansion convention and explain whether the goal is tested when generated or when selected for expansion.

For A-star, begin with a simple heuristic such as straight-line distance in a map with coordinates. Verify that the heuristic does not overestimate the true remaining shortest-path cost for the test map. Then deliberately create an overestimating heuristic and observe whether the returned path remains optimal. Explain the result rather than merely reporting that the output changed.

Finally, modify the state representation. Add a binary variable indicating whether an access door is open. Add a coarse battery category: high, medium or low. Show how the additional state variables change reachability and planning. Discuss whether the added detail is necessary for the task and what further state information might be required for a deployable robot.

11. Common errors

One common error is confusing the search tree with the state-space graph. This leads to duplicate work and failure to detect cycles. Draw both structures explicitly when debugging.

A second error is declaring an algorithm optimal without stating its assumptions. Breadth-first search is optimal for equal step costs, not arbitrary costs. A-star's guarantee depends on properties of the heuristic and the implementation. Uniform-cost search relies on appropriate non-negative costs.

A third error is marking a node visited too early. In cost-sensitive search, a cheaper path to a previously discovered state may later be found. The implementation must allow a state's recorded best cost to be updated when appropriate.

A fourth error is using a heuristic that measures the wrong quantity. Straight-line distance may be suitable for a distance objective but not necessarily for time, energy or risk. The heuristic should correspond to the cost being minimized and preserve any desired guarantees.

A fifth error is using a state that omits information needed to predict the future. If door status changes available actions, door status may belong in the state. If only location is stored, two situations with different access conditions may be incorrectly treated as identical.

12. Review and assignment

Answer the following questions. What five elements define a basic search problem? Why can the same world state appear multiple times in a search tree? Under what cost assumption is breadth-first search optimal? How does uniform-cost search differ from depth-first search? What does an admissible A-star heuristic guarantee under standard assumptions? Why might a route planner need battery level or door status in its state representation?

For the assignment, submit a short report and working implementation. Define a small robot navigation task, draw its state-space graph, identify the initial state and goal test, specify action costs, and implement at least three search strategies. Include a table of path quality and computational effort. Explain one representation choice and one failure mode. Your report should distinguish observed results from algorithmic guarantees.

Module summary

We formulated search problems using an initial state, actions, transition model, goal test and path cost. We distinguished the underlying state-space graph from the search tree generated during exploration. We compared breadth-first search, depth-first search, uniform-cost search and A-star, and discussed completeness, optimality, memory and heuristic assumptions. Most importantly, we saw that an algorithm cannot rescue an incorrect problem formulation. The state representation and cost model determine what the algorithm can find and what it considers a good solution.